

Autonomous capture of agile flying objects using UAVs: The MBZIRC 2020 challenge

Matouš Vrba^{*}, Yurii Stasinchuk, Tomáš Báča, Vojtěch Spurný, Matěj Petrlik, Daniel Heřt, David Žaitlík, Martin Saska

Multi-robot Systems Group (MRS), Faculty of Electrical Engineering, Czech Technical University in Prague, Technická 2, Prague 6, Czech Republic

ARTICLE INFO

Article history:

Received 18 August 2021

Received in revised form 6 December 2021

Accepted 7 December 2021

Available online 17 December 2021

Keywords:

Unmanned aerial systems

Machine perception

Mobile robotics

Aerial safety

ABSTRACT

In this paper, a novel approach for autonomously catching fast flying objects is presented, as inspired by the Mohamed Bin Zayed International Robotics Challenge (MBZIRC) 2020. In this competition, an autonomous Unmanned Aerial Vehicle (UAV) was used to intercept a ball carried by a fast flying drone. The presented solution utilizes a 3D LiDAR sensor for quick and robust target detection. The trajectory of the target is estimated and predicted to select a suitable interception position. The interceptor UAV is navigated into the interception position to safely approach the target. The interception position is frequently being adjusted based on the updated estimation and prediction of the target's motion to ensure that the ball is caught in the dedicated onboard net. After a successful interception is detected, the UAV lands in a designated landing area. The proposed concept was intensively tested and refined in demanding outdoor conditions with strong winds and varying perception conditions to achieve the robustness required by both the demanding application and the competition. In the MBZIRC 2020 competition, our solution scored second place in Challenge 1 and first place in a combined Grand Challenge. This manuscript will provide a detailed description of the applied methods and an evaluation of our approach with data collected from real-world experiments. In addition, we present achievements of our R&D towards the transition from the MBZIRC competition to an autonomous drone interceptor, which was the main motivation of this challenge.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

The problem of drone interception using an autonomous UAV is gaining increased attention in the robotics community due to the importance of protecting critical infrastructure and no-fly zones against non authorized aerial vehicles [1–5]. One of the tasks being explored in this field of research is the detection of intruder drones within a designated perimeter by onboard sensors of a drone hunting UAV. When the target is detected, it needs to be precisely localized and tracked. An onboard mission planning system manages autonomous target following and interception thereafter. For such autonomous drone hunting systems, as well as for Challenge 1 of the MBZIRC 2020 competition, the key properties include high reliability and fast mission accomplishment.

Challenge 1 of MBZIRC 2020 consisted of two tasks that were carried out autonomously by three cooperating UAVs. In one task, a team of UAVs was required to localize and physically eliminate a set of static targets represented by balloons in a given area. The other task was to catch a target ball suspended below a fast-moving drone following an unknown trajectory in the area. Both tasks are motivated by the drone hunting scenario, but in this paper we focus only on the more challenging task of catching the ball, which is also more relevant to real drone hunting missions.

Our research group has advanced expertise in the autonomous drone interception field [1,2], and the proposed system benefited from this long-term experience. Although the MBZIRC scenario differs from general drone hunting tasks, it introduced new challenges, due mainly to the small size and aggressive movement of the target which required new approaches. Our solution achieved first place in the Grand Challenge and second place in Challenge 1.¹ To put in perspective the complexity of the assignment, the presented system was only one of four solutions among 22 expert competitors (selected in the qualification phases from the best robotic groups from around the world) who were able to autonomously detach the target ball from the carrier drone.

^{*} Corresponding author.

E-mail addresses: matous.vrba@fel.cvut.cz (M. Vrba), stasiyur@fel.cvut.cz (Y. Stasinchuk), tomas.baca@fel.cvut.cz (T. Báča), vojtech.spurny@fel.cvut.cz (V. Spurný), matej.petrlik@fel.cvut.cz (M. Petrlik), daniel.hert@fel.cvut.cz (D. Heřt), david.zaitlik@fel.cvut.cz (D. Žaitlík), martin.saska@fel.cvut.cz (M. Saska).

¹ mbzirc.com/winning-teams/2020.

There are multiple factors contributing to the difficulty of the assignment. The target has small dimensions (the ball was only 11 cm in diameter) and moves very fast (up to 8 m s^{-1}) on a relatively large-scale trajectory, making it difficult to be detected and tracked. When the target is detected, the catcher UAV must react in an agile manner in order to catch it before it flies outside the manipulation range of the catcher. Only sensors placed onboard the competing UAVs may be used for detection of the target and for self-localization of the UAVs. Because the size (and thus, the effective payload size and weight) of the catcher UAV is constrained by the competition rules, only relatively lightweight onboard sensors and equipment can be deployed. Another complication is that the catcher UAV must not make physical contact with the ball-carrying drone, leaving minimal room for error when intercepting the ball. All of these requirements go far beyond the capabilities of existing drone hunter aerial systems, as is discussed in Section 1.2. As shown in the section describing technology transfer of scientific achievements obtained for the MBZIRC competition into an industrial project (Section 5), these requirements were motivated by achieving better performance and robustness of the autonomous drone hunting solution.

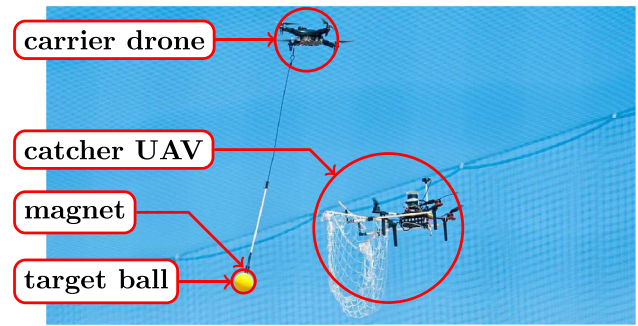
The system described in this paper is built upon our existing hardware and software UAV platform, which is the result of long-term research in aerial robotics, UAV control, planning, and multi-robot systems. To further demonstrate its robustness and universality, our team also used this platform to successfully solve the other two challenges of the MBZIRC 2020. The key components of the hardware and software platform are described in Section 2. To successfully execute the ball-catching mission, a novel approach had to be devised. This approach consisted of three main components:

- target detection and velocity estimation,
- target trajectory estimation,
- and interception planning.

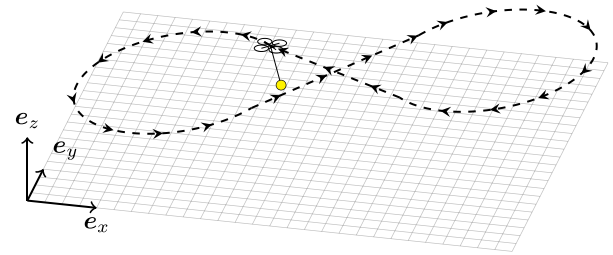
For the detection and estimation of the target's velocity, a detection algorithm was developed using a 3D point cloud produced by an onboard LiDAR sensor. This is a novel approach that has never been used in the available drone hunting solutions. Future positions of the ball are predicted based on the assumption of periodicity of its trajectory in the competition. In real drone hunting scenarios, multiple pre-learned patterns of common drone behaviors may be applied in a similar way. Several shape-fitting methods are utilized to estimate important segments of the trajectory. A suitable position to intercept the target and catch the ball is selected based on the estimated trajectory segments. The whole interception method is described in detail in Section 3.

1.1. Problem definition

In this section, we describe in detail the relevant parts of Challenge 1. In the ball catching task of Challenge 1, a soft yellow ball with a diameter of 11 cm is attached magnetically to a 1.45 m long wire hanging from a DJI F450 drone (see Fig. 1(a)). The drone flies in a periodic trajectory in the shape of the eight character ('8'), oriented randomly in 3D space (see Fig. 1(b)). Initial speed of the carrier drone is 8 m s^{-1} , which is decreased to 5 m s^{-1} after 450 s of flight. The mission is to use a UAV to mechanically detach the ball from the wire and deliver it to a designated landing area without making physical contact between the catcher UAV and the carrier drone. The catcher UAV is restricted to maximum dimensions of $1.2 \text{ m} \times 1.2 \text{ m} \times 0.5 \text{ m}$ and a maximum speed of 8.3 m s^{-1} . The landing area is $10 \text{ m} \times 10 \text{ m}$ and the total volume of the challenge arena is $100 \text{ m} \times 60 \text{ m} \times 20 \text{ m}$. Apart from Challenge 1, the ball catching task was included in the Grand Challenge where tasks from all three separate challenges of the competition were solved simultaneously.



(a) A photo of the carrier drone with the target ball attached and our catcher UAV platform.



(b) A schematic of an example trajectory of the carrier drone with the target ball.

Fig. 1. Illustration of the ball catching task.

1.2. Related work

Although the ball catching challenge of the MBZIRC 2020 is a very specific task, several related problems have been tackled and published by the robotics community. There are several works dealing directly with ball catching or ball evasion, such as [6–9]. These focus on various elements of the problem, mostly on prediction of the future ball trajectory with simplifying assumptions applied to the other elements (e.g. using motion-capture systems for UAV and ball localization, or deploying the system only in simulation). All of these works deal with a thrown or a falling ball, which follows a simple ballistic trajectory due to gravity that is easy to estimate and predict. In our case, the target ball follows an unknown trajectory at high speed in a large outdoor area. There is no possibility of using a motion-capture system or other simplifications of the problem. This makes the MBZIRC Challenge 1 especially challenging when compared to situations of falling or thrown balls.

The ball catching scenario was inspired by the problem of autonomous interception of intruding drones for the purpose of guarding vital infrastructure, private property, and other no-fly zones. Commercially available products for the detection of unauthorized drones do exist, such as DeDrone or SkyWall.² These solutions primarily rely on the installation of static cameras, RADARs, and pilot-drone communication receivers installed around the protected perimeter. For drone interdiction, Global Navigation Satellite System (GNSS) or Radio-Control jamming is often used, which is only applicable to cases where the intruder relies on these signals to operate, is unprotected from jamming, and if the jammers are legal for use. Several solutions provide manual or semi-autonomous physical interdiction, but a reliable autonomous aerial interception system is yet to be conclusively

² dedrone.com, openworkengineering.com/skywall.

verified. This topic has lately gained increased attention from the robotics community and much new research tackles this problem. Some of the approaches utilize radar [10,11], acoustic [12,13] or mixed [14] sensors for detection of the intruder, which is also common in the commercial products. However, such sensors are usually large in size, heavy, and therefore exclusively ground-based. For the autonomous interception scenario, the sensor must be onboard the interceptor aerial system to provide fast feedback and precise localization of the target. This disqualifies most of such approaches.

Most of the existing approaches of dynamic target tracking utilize computer vision methods for detection and localization of the target. To our best knowledge, all other teams in the MBZIRC 2020 Competition relied on computer vision to solve the Challenge 1 tasks [5,15–17]. Specifically, most of these approaches relied on Convolutional Neural Networks (CNNs) for detecting the target using a monocular RGB camera, which is a common theme found in works tackling general drone detection [18–20]. Monocular camera sensors can be small and lightweight and are generally suitable for deployment onboard UAVs. However, even though specialized CNN hardware has recently improved, robust and precise CNNs still lack a good processing frame-rate and introduce unwanted delays to the system when running without a large and heavy dedicated GPU, which is unfeasible onboard a UAV. Other drone detection approaches employ more traditional computer vision-based methods, such as in [21,22]. We had tested the use of a monocular RGB camera for detection of the target in the competition, but concluded that the effective detection range is too limited for the task as the target is too small. This is also discussed in the experimental evaluation in Section 4. This disadvantage may be mitigated by using a PTZ (pan-tilt-zoom) camera system, as was used by the Beijing Institute of Technology team in both this and the previous competition [23]. Even so, general drone detection from a monocular RGB camera proves to be difficult when the target drone is in front of a structured background (e.g. a forest) due to the relatively small size and low contrast-colors of most available drones, as was reported in [1]. Because our aim was to develop an approach applicable to the general problem of detection for drone hunting missions and not specialized to the specific competition scenario, we have avoided using a monocular RGB camera-based detection method.

Some researchers propose to use a depthmap image from a stereo camera for onboard drone detection. Because a drone is a flying object, it produces a distinct shape in the depthmap image, which may be detected using a specialized algorithm [1] or using a modified CNN originally intended for detection from an RGB image [24]. This approach mitigates the problem of detection from an RGB camera, since the drone is a prominent feature in the depthmap if it is closer than the background, even if the drone would be hard to spot in the RGB image of the same scene. Additionally, the depthmap provides a good measurement of the distance of the target, which is problematic when using only a monocular camera. However, the range of the depthmap from a stereo camera depends on the stereo baseline, which can be limiting especially in the case of small aerial vehicles with limited payload capacity. In [2], a combination of the monocular and depthmap based approaches is proposed to mitigate the limitations of both of these methods. For the target detection in the competition, we have tested using the Realsense D435 stereo camera which is relatively small and lightweight, but the range at which the target could be detected was too short for successful interception of the target. A larger stereo camera could not be used due to size constraints on the UAVs used in the competition.

Because of the limitations of vision-based methods that we had explored in preliminary experiments, a different detection method was chosen for our solution. After various tests, we

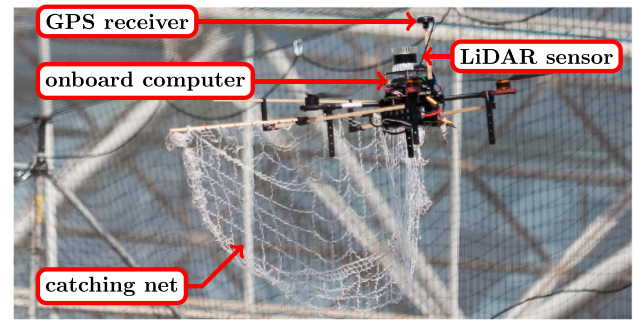


Fig. 2. A photo of the ball catching UAV platform.

converged on using a 3D LiDAR sensor for the detection. To improve the performance, instead of looking only for the target ball, both the drone and the ball are detected as it provides a larger target than the ball alone. Most of the research in detection of drones with a 3D LiDAR focuses on ground-based systems, such as in [25–27]. This greatly simplifies the problem as the LiDAR can be stationary and consecutive pointclouds do not require alignment. LiDAR onboard UAVs is mostly used for forest mapping [28], autonomous inspection of infrastructure [29], or simultaneous localization and mapping (SLAM) [30]. However, scientific publications on autonomous drone detection from a LiDAR onboard a UAV is sparse. In [31], an evaluation of a RADAR and a LiDAR sensor for onboard drone detection is provided. The authors conclude that both these technologies are suitable for such tasks, but autonomous detection is not tackled.

A similar approach to our lurking interception strategy (described in detail in Section 3) was used by several teams in the previous MBZIRC Competition in 2017, where one of the challenges required a UAV to land on a moving ground vehicle [23,32–34]. The vehicle followed a periodic trajectory with a speed of approximately 4 m s^{-1} . The landing platform was highlighted with a visual marker, which is a similar to the ball catching task regarding mission control, estimation, and planning. The ball catching task was designed as the next step of this challenge and is reflected by its higher complexity. Compared to the ground task, the target ball moves significantly faster (8 m s^{-1}) along a trajectory in 3D space and is much smaller without any easily detected markers. Additionally, physical interaction with the target must also be more precise in order to intercept the ball without hitting the carrier drone. Therefore, new methods had to be developed to solve the ball catching task.

2. UAV platform

The ball catching UAV (see Fig. 2) used in the MBZIRC 2020 competition is built upon our basic UAV platform [33,35–37]. This includes the hardware (the frame, motors, sensors, computer, etc.) as well as the software (trajectory generation and control algorithms, sensor drivers, etc.). An overview schematic of the complete ball catching UAV platform is seen in Fig. 3.

2.1. Hardware

The basis of the hardware platform includes the Tarot T650 quadrotor frame with the PixHawk 4 flight controller and an Intel NUC onboard computer. The PixHawk 4 includes sensors required to stabilize a multirotor craft: a gyroscope, an accelerometer, a barometer, and a GNSS with a magnetometer. The PixHawk fuses data from these sensors to obtain a pose and velocity estimate of the UAV, which are then transmitted to the onboard computer, and also controls the Electronic Speed Controllers (ESCs)

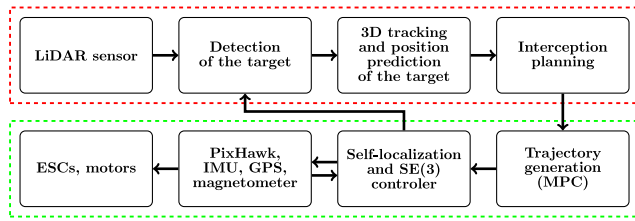


Fig. 3. A schematic of the ball catching UAV. Components marked with the green dashed line are parts of our general-purpose UAV platform, whereas components in the red dashed box have been developed and added specifically for the purpose of ball catching and are the main focus of this report. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

that drive the brushless motors propelling the UAV based on attitude commands from the onboard computer. A 3D LiDAR sensor (Ouster OS1-32, Gen. 1) provides data for a detection algorithm to detect the target (parameters of the LiDAR used in the competition are listed in Table 1). To catch and hold the target, a net mechanism was designed and attached to the catcher UAV (see Fig. 2). The sensors, batteries, onboard computer, etc., are mounted on the main frame using custom 3D printed parts. The onboard computer is a NUC8i7BEH with an Intel i7-8559U CPU and 8 GB of RAM.

2.2. Software

All high-level control, planning, data processing, etc., runs on the onboard computer, including the interception algorithms described in Section 3. A self-localization algorithm based on the Kalman filter fuses data from the PixHawk estimator and a laser rangefinder (Garmin LiDAR-Lite v3), which is used for height estimation. If any other sources of the UAV position are available (such as optical flow, RTK-GNSS, or SLAM), they may also be fused by this algorithm. However, in the case described in this paper, only GNSS, magnetometer, IMU, and the laser rangefinder were used for self-localization of the UAV. The self-localization system is described in more detail in [36]. Some teams used RTK-GNSS to improve the precision of the UAV localization, but we avoided this solution as it was penalized in the competition rules by a 25% reduction in points.

A geometric tracking controller on SE(3) [38] ensures agile and precise following of the setpoint trajectory, which is generated using a Model Predictive Control (MPC) tracker [35] based on commands from the application-specific high-level planner. The MPC tracker ensures that the setpoints for the SE(3) controller conform to the UAV dynamics by modifying the input from the planner based on a linear system model and set constraints to speed, acceleration, jerk, and snap. A detailed description of the control system is provided in [37].

A target detection algorithm (described in Section 3.2) uses information from the self-localization system of the UAV to transform pointclouds from the onboard LiDAR sensor to a static global frame, which is then used to detect and localize the target drone with the ball. The detections are processed by a 3D tracking and prediction algorithm, which in turn provides data for a high-level interception planner. The output of the planner is a trajectory setpoint that is executed by the UAV control pipeline.

All software described in this paper runs on the NUC onboard computer under the Ubuntu 18.04 LTS operating system and ROS Melodic middleware software for robots [39]. ROS provides an API for communication between the various software elements on the platform, interfacing with the sensors, time synchronization of the different algorithms, as well as tools for visualization of data, etc.

Table 1

Parameters of the Ouster OS1-32 Gen. 1 3D LiDAR.

Horizontal resolution	Vertical resolution	Vertical FOV	Scan rate	Maximal range
2048	32	$\pm 16.6^\circ$	10 Hz	120 m

3. Autonomous interception method

After studying the task in simulations and preliminary experiments while taking into account the competition rules, we decided for a “lurk & catch” interception strategy. The aim of this strategy is to select a suitable “lurking” spot where the catcher UAV waits for the target to fly itself into the onboard catching net (as in Fig. 1(a)). This strategy proved to be more robust than an active chasing strategy given our platform’s limited dynamics when compared to the target.

The position of the target is detected in the LiDAR data based on an a priori known flight area of the target. The operational area is defined as a 3D volume in a static global frame and was pre-measured with a GNSS before the start of the competition.³ According to the competition rules, the trajectory of the ball-carrying drone is periodic and has a shape of the eight symbol (“8”), whose scale and orientation is randomized for each trial of the competition. Based on this assumption, a plane is fitted to a set of detections to obtain the plane in which the trajectory of the target lies. Two line segments are fitted to the detections to find a suitable lurking spot where dynamics of the target can be simplified to a constant velocity movement along a line. The result is illustrated in Fig. 5.

After the lurking spot is selected, the catcher UAV relocates there and waits for the target to approach it. The future trajectory of the target is predicted using a Kalman Filter (KF) and a constant velocity movement model. This prediction is used to adjust the pose of the catcher UAV. A camera is used to check whether the ball is present in the catching net. If the ball is detected in the net, the mission is finished and the catcher UAV lands in the designated landing area.

The target detection and velocity estimation algorithms are executed with each new LiDAR scan. For the OS1-32 sensor used in the competition, this means a sampling period of 10 Hz. The estimation, prediction, planning, and mission control algorithms are executed with a constant 10 Hz period. All of these algorithms have a negligible computation time (in the order of milliseconds) when compared to the execution period (100 ms). Parameters of the algorithms presented in this section are listed in Table 2. These were chosen based on realistic simulations and preliminary experiments (see Section 4). The code for all methods and algorithms presented in this paper is available online.^{4,5,6}

3.1. Planning and mission control

The mission is controlled by a state machine, which is illustrated in Fig. 4. For the general autonomous interception scenario, the state machine would have to be modified to include confirmation of the target and active chasing instead of the lurking state. The state machine is activated by the UAV control system (described in Section 2.2) after take-off of the catcher UAV, and deactivated either after the mission is finished, or manually by the safety pilot. The states are:

³ For safety reasons, all movement of the catcher UAV is limited to a safety area, which was obtained in a similar manner.

⁴ github.com/ctu-mrs/uav_detect (branch *mbzirc*).

⁵ github.com/ctu-mrs/mbzirc2020_ball_filter.

⁶ github.com/ctu-mrs/mbzirc2020_ball_planner.

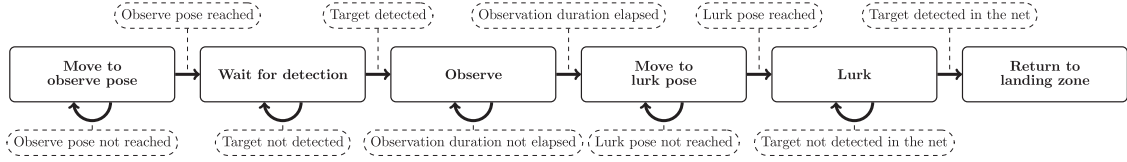


Fig. 4. A schematic of the mission control state machine.

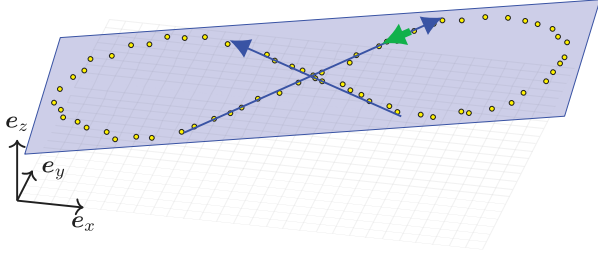


Fig. 5. An illustration of detections (marked with yellow circles) with the corresponding fitted plane (marked blue) and line segments (blue arrows). The selected lurking pose is marked with a green arrow. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 2

Parameters of the presented algorithms and their respective values as used in the competition.

Symbol	Meaning	value
Detection classification parameters		
$h_{A,max}$	max. cluster height for case A	2.5 m
$r_{A,max}$	max. cluster radius for case A	2 m
d_{close}	min. cluster dist. for case B	9 m
$r_{B,max}$	max. cluster radius for case B	0.28 m
$h_{B,C,max}$	max. height for cases B and C	1 m
Position estimation parameters		
$\sigma_{A,xy}^2, \sigma_{A,z}^2$	cov. parameters for case A	0.01 m ² , 0.03 m ²
$\sigma_{B,xy}^2, \sigma_{B,z}^2$	cov. parameters for case B	0.05 m ² , 0.05 m ²
$\sigma_{C,xy}^2, \sigma_{C,z}^2$	cov. parameters for case C	0.1 m ² , 0.1 m ²
Velocity estimation parameters		
r_{near}	nearby detection radius	5 m
Δt	max. detections' age	3 s
$N_{vel,min}$	min. number of detections	2
Δs_{max}	max. error from ground-truth	3.8 m s ⁻¹
Lurking pose selection parameters		
$N_{fit,max}$	max. detections considered	100
$N_{fit,min}$	min. detections for valid plane	10
$N_{inl,min}$	min. detections for valid line	10
s_{back}	line endpoint back-off distance	4 m
Trajectory prediction parameters		
$Z_{\bar{v}}$	cov. of velocity measurements	$0.4 \cdot \mathbf{I}_{3 \times 3}$
Ξ	cov. of process noise	$\begin{bmatrix} 0.01 \cdot \mathbf{I}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & 0.05 \cdot \mathbf{I}_{3 \times 3} \end{bmatrix}$
$N_{init,min}$	min. data. to initialize KF	5
Δt_{max}	max. age of KF estimate	3 s
$t_{horizon}$	trajectory prediction horizon	4 s
d_{snap}	max. line distance to snap	1 m

1. **Move to observe pose:** The catcher UAV flies to a predefined observation position that is selected manually in the GNSS-measured map of the arena. When the UAV reaches the position, the next state is activated.
2. **Wait for detection:** The UAV flies up and down at the observation pose in a predefined height interval, which is selected based on the allowed height range of the target in the competition rules. Once the target is detected, an observation timer is started and the next state is activated.

3. **Observe:** Height of the catcher UAV is adjusted based on the latest detection to maximize detection probability. This state serves to collect enough detections for the lurking pose selection algorithm, described in Section 3.3.1. After the observation timer reaches the observation duration t_{obs} and if a lurking pose is available, the next state is activated. When the state transition occurs, the current lurking pose $W_{[k]}$ is fixed as $W^* \equiv \{u_{lurk}^*, \varphi_{lurk}^*\} = W_{[k]}$.
4. **Move to lurk pose:** The catcher UAV flies to the lurking pose selected during the previous state. When the UAV reaches the position, the next state is activated.
5. **Lurk:** Pose of the UAV is adjusted according to the target's future trajectory, which is predicted by the algorithm from Section 3.3.2. The pose adjustment strategy is described further in this section. The next state is activated if the target ball is detected in the catching net.
6. **Return to landing zone:** The UAV returns to the landing zone at the designated GNSS coordinates, lands, and ends the mission.

3.2. Target detection and velocity estimation

The target detection algorithm is applied to each input LiDAR scan. The algorithm for detection and velocity estimation of the target has four main steps. These are:

1. detection of point clusters of interest,
2. cluster classification,
3. position estimation of the target,
4. velocity estimation of the target.

In the first step, potential detection candidates consisting of point clusters are located in the input pointcloud using apriori knowledge of the target's approximate flight area and size. The detection candidates are classified based on their shape, and the most likely valid detection is picked. The class of the detection is used for estimating the position of the target ball and the corresponding covariance. A voxel map of recent detections is used to estimate approximate velocity of the target and is updated with each detection.

3.2.1. Detection of point clusters of interest

Since most of the available methods use 3D LiDARs for robot localization [40], environment mapping [41], exploration [42], or obstacle detection [26,43], a new approach of point cloud processing is required for this specific application. In this case, a relative position of an object in the scan is required, as is common in obstacle detection and avoidance methods. However, the novelty lies in the necessity to identify a particular object of interest and estimate its state, which is needed for an effective and safe approach and successful grasp of the ball as the catcher UAV cannot touch the carrier. In addition, motion estimation of the object is needed to allow for the dynamic grasping action. This enables using the proposed method for general aerial manipulation with fast moving objects.

It is assumed that the target and the catcher UAV are the only objects present in the interception operational area. The pointcloud from each scan of the onboard LiDAR is transformed

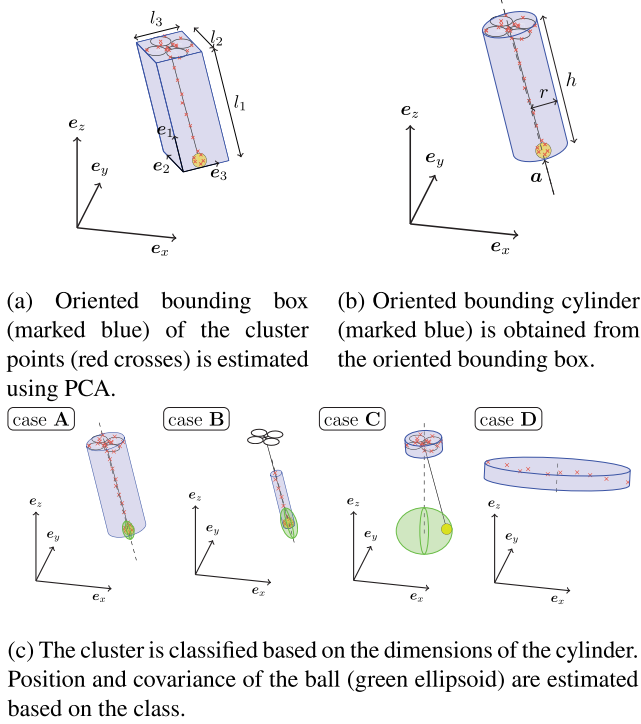


Fig. 6. Steps of the classification process of a point cluster.

to a static global frame using the current pose of the UAV, which is obtained from its self-localization system (as described in Section 2.2). Points corresponding to the catcher UAV are removed from the pointcloud based on known dimensions of the UAV and its position. The remaining points within the operational area are separated into clusters using a euclidean clustering algorithm. Clusters with the number of points $N \geq N_{\min}$ are considered as candidates for a detection.

The target ball and the drone carrying the ball are detected as one object together, which makes the detection target larger and easier to detect at longer distances. In addition, considering the entire system as a compact object allows for estimation of a relative position of the required contact point (the ball) and the carrier (which must be avoided) using Principal Component Analysis (PCA), as is required for any task of cooperative aerial manipulation.

3.2.2. Cluster classification

When detecting a small, fast flying target attached to another object using a real sensor with a limited Field Of View (FOV), one problem that occurs is that the whole system will not always be contained in a sensor scan. In such a case, even though a part of the system is located outside the sensor FOV, the captured data may still provide useful information. However, to reliably extract the information, this case must be detected. To address this problem, a cluster classification method was designed that classifies different variants of such cases. The cluster classification can also be extended, even for multiple different objects of interest, if required by a given application.

Each detection candidate is classified as one of four special cases (see Fig. 6(c)), which were identified in real 3D data collected in preliminary experiments. The position of the target ball and its covariance is calculated differently for each case. The four cases are:

A The cluster contains points from both the carrier drone and the target ball (possibly also the rod). This case is common

when the target is within the range of 3 m to 20 m, where laser rays of the LiDAR sensor hit both the drone and the ball with high probability. This situation is illustrated in Fig. 6(c), first image from the left.

B The cluster contains points from the target ball and possibly also part of the rod. This case occurs when the target is very close to the LiDAR sensor and the carrier drone is outside the vertical FOV of the LiDAR. This situation is illustrated in Fig. 6(c), second image from the left.

C The cluster contains points from the target carrying drone and possibly also part of the rod. This case often occurs when the target is far from the LiDAR sensor (beyond 10 m), since the drone is larger than the target ball and rays of the LiDAR therefore hit it with a higher probability. This situation is illustrated in Fig. 6(c), third image from the left.

D The cluster contains points from a non-target object. This may happen, e.g., when the GNSS drifts significantly and a part of the arena bounding net is considered inside the operational area. With this cluster class, the proposed system is general enough to be used in any dynamic aerial manipulation task even in environments with obstacles. This situation is illustrated in Fig. 6(c), last image from the left.

Case **A** was the most common in the competition and provides the most reliable estimation of the target position. It is important to differentiate between cases **B** and **C** to get a good position estimation of the target ball when the target is either close or far from the catcher UAV. Case **D** serves to reject false positives.

A candidate cluster is classified based on the dimensions of its minimal oriented bounding cylinder. It is assumed that the cylinder is oriented vertically with a small tilt caused by speed and acceleration of the target. The cylinder is calculated from a minimal Oriented Bounding Box (OBB), which is estimated using PCA. The classification process is described in Algorithm 1 and illustrated in Fig. 6.

Algorithm 1 Classification of a detection candidate cluster

```

1: Input:
2:    $C = \{p_1, p_2, \dots, p_N\}$   $\triangleright$  a set of points in the cluster
3:    $p_{UAV}$   $\triangleright$  position of the catcher UAV
4: Output:
5:   class  $\in \{A, B, C, D\}$   $\triangleright$  class of the cluster
6:    $R \in \mathbb{R}^{3 \times 3}$   $\triangleright$  orientation of the cluster
7:  $\triangleright$  find axes, the corresponding edge lengths, and center of the OBB
8:  $\{R = [\vec{e}_1 \ \vec{e}_2 \ \vec{e}_3], \mathcal{L} = \{l_1, l_2, l_3\}, p\} := \text{PCA}(C)$ 
9:  $\triangleright$  find index of the most vertically oriented axis
10:  $i := \arg\max_{i \in \{1,2,3\}} |\vec{e}_i^T \cdot \vec{e}_z|$ 
11:  $\triangleright$  rearrange the columns of  $R$  to put the selected axis in the third column
12: if  $i \neq 3$  then
13:    $R := [\vec{e}_3 \ \vec{e}_i \times \vec{e}_3 \ \vec{e}_i]$ 
14:  $\triangleright$  height of the cylinder is the length of the corresponding edge
15:  $h := l_i$ 
16:  $\triangleright$  radius of the cylinder is the largest remaining edge length
17:  $r := \max(\{l_1, l_2, l_3\} \setminus l_i)$ 
18:  $\triangleright$  calculate distance of the cluster from the catcher UAV
19:  $d := \|p - p_{UAV}\|$ 
20:  $\triangleright$  select the class based on the obtained properties of the cluster
21: if  $r \in (r_{B, \max}, r_{A, \max}]$  and  $h \in (h_{B, \max}, h_{A, \max}]$  then
22:   class := A
23: if  $r < r_{B, \max}$  and  $d < d_{\text{close}}$  and  $h \leq h_{B, \max}$  then
24:   class := B
25: if  $(r \geq r_{B, \max} \text{ or } d \geq d_{\text{close}})$  and  $h \leq h_{B, \max}$  then
26:   class := C
27: if  $h > h_{A, \max}$  or  $r > r_{A, \max}$  then
28:   class := D

```

3.2.3. Position estimation of the target

Clusters classified as **D** are discarded as false positives. From the remainder, the cluster with the highest number of points

is selected. Position $\mathbf{b}$ and position covariance $\mathbf{Z}_b$ of the target is estimated from the selected cluster based on its class using Algorithm 2. $\mathbf{Z}_b$ represents the uncertainty of the position measurement and is different for the various cases, reflecting different levels of available information about the target (illustrated in Fig. 6(c)).

Algorithm 2 Position estimation of a classified cluster

```

1: Input:
2:    $\mathcal{C} = \{\mathbf{p}_1, \mathbf{p}_2 \dots \mathbf{p}_N\}$   $\triangleright$  a set of points in the cluster
3:   class  $\triangleright$  class of the cluster
4:    $\mathbf{R} \in \mathbb{R}^{3 \times 3}$   $\triangleright$  orientation of the cluster
5: Output:
6:    $\mathbf{b} \in \mathbb{R}^3$   $\triangleright$  an estimated position of the ball
7:    $\mathbf{Z}_b \in \mathbb{R}^{3 \times 3}$   $\triangleright$  a covariance of the position estimate
8: switch class do
9:   case A
10:     $\triangleright$  estimate the ball position as the lowest point in  $\mathcal{C}$ 
11:     $\mathbf{b} := \operatorname{argmin}_{\mathbf{p}_i \in \mathcal{C}} (\mathbf{p}_i^\top \cdot \tilde{\mathbf{e}}_z)$ 
12:     $\mathbf{Z}_b := \mathbf{R} \begin{bmatrix} \sigma_{A,xy}^2 & 0 & 0 \\ 0 & \sigma_{A,xy}^2 & 0 \\ 0 & 0 & \sigma_{A,z}^2 \end{bmatrix} \mathbf{R}^\top$ 
13:   case B
14:     $\triangleright$  estimate the ball position as the lowest point in  $\mathcal{C}$ 
15:     $\mathbf{b} := \operatorname{argmin}_{\mathbf{p}_i \in \mathcal{C}} (\mathbf{p}_i^\top \cdot \tilde{\mathbf{e}}_z)$ 
16:     $\mathbf{Z}_b := \mathbf{R} \begin{bmatrix} \sigma_{B,xy}^2 & 0 & 0 \\ 0 & \sigma_{B,xy}^2 & 0 \\ 0 & 0 & \sigma_{B,z}^2 \end{bmatrix} \mathbf{R}^\top$ 
17:   case C
18:     $\triangleright$  find the point in  $\mathcal{C}$  with the highest z-coordinate
19:     $\mathbf{b}_{\text{top}} := \operatorname{argmax}_{\mathbf{p}_i \in \mathcal{C}} (\mathbf{p}_i^\top \cdot \tilde{\mathbf{e}}_z)$ 
20:     $\triangleright$  estimate the position as a point below  $\mathbf{b}_{\text{top}}$  with  $l_{\text{pole}}$  offset
21:     $\mathbf{b} := \mathbf{b}_{\text{top}} - l_{\text{pole}} \tilde{\mathbf{e}}_z$ 
22:     $\mathbf{Z}_b := \begin{bmatrix} \sigma_{C,xy}^2 & 0 & 0 \\ 0 & \sigma_{C,xy}^2 & 0 \\ 0 & 0 & \sigma_{C,z}^2 \end{bmatrix}$ 

```

3.2.4. Velocity estimation of the target

Another challenging task that was solved was the estimation of the target's motion, which may be used in any application of dynamic aerial manipulation with fast moving objects and in intruder drone interception. In this scenario, both the target and the 3D sensor carried by the UAV are moving independent to each other. This requires integration and synchronization of the UAV state estimation and processed 3D data in time, while taking into account often incomplete sensory information of the flying object being observed.

The velocity $\tilde{\mathbf{v}}$ of the target is estimated in addition to its position $\mathbf{b}$ if a sufficient number of reliable past detections is available. The velocity estimate serves to improve performance of the trajectory prediction algorithm, described in Section 3.3.2. Each detected position of the target is added to an octree data structure, together with a corresponding time-stamp. To estimate velocity of the target at position $\mathbf{b}$ and time t , detections within radius r_{near} from $\mathbf{b}$ and with a time-stamp newer than $t - \Delta t$ are retrieved from the octree. If less than $N_{\text{vel,min}}$ points meet these conditions, no velocity is estimated. Otherwise, a line is fitted to these points using the Random Sample Consensus (RANSAC) algorithm. Inliers of the RANSAC-fitted line are sorted by their respective time-stamps, and a mean velocity $\tilde{\mathbf{v}}$ is calculated from these time-stamped points.

Note, that the ground-truth speed of the target s_{gt} was specified by the competition rules. If $\|\tilde{\mathbf{v}}\| - s_{\text{gt}} > \Delta s_{\text{max}}$ holds true, the velocity estimate is discarded to filter out invalid estimates. Otherwise, $\tilde{\mathbf{v}}$ is scaled so that $\|\tilde{\mathbf{v}}\| = s_{\text{gt}}$. The threshold Δs_{max} is also used to reset an estimate of the target's state, as described in Section 3.3.2. Its value was determined empirically during preliminary experiments to be quite lenient, as the estimation of the velocity direction proved to be reliable even when the magnitude was off.

3.3. Trajectory estimation

The trajectory of the target is estimated for two purposes with different requirements:

- selection of the lurking spot,
- online adjustment of the lurking pose.

The algorithm for lurking pose selection locates a specific segment of the periodic trajectory, which was empirically found to be the most reliable for the interception. This algorithm is described in Section 3.3.1. After the lurking pose is selected and the interception action begins, the pose of the catcher UAV is adjusted using the latest predicted trajectory of the target to accommodate for GNSS drift of the UAV position. The trajectory prediction technique is described in Section 3.3.2. Both these algorithms utilize output of the detection algorithm presented in the previous section.

Henceforth, variables from the i th time-step will be denoted with a lower index “[i]” (such as $\mathbf{b}_{[i]}$ for the i th detected position of the target), and the current time-step will be denoted k when required.

3.3.1. Lurking pose selection

The procedure described in this section is specifically designed for the MBZIRC 2020 task in order to optimize system performance. In a general drone hunting scenario, the lurking pose would be selected simply on a predicted trajectory of the target, while taking into account dynamic constraints of the hunter UAV.

Based on the specifications of the competition rules, it is assumed that the trajectory of the target is periodic, planar, and contains two intersecting line segments (see Fig. 1(b)). After experimental analyses of the probability of success for the given task in the Gazebo simulator, the most suitable waiting point for our interception strategy was identified near the end of either of these line segments. The line segments are considered oriented according to the direction of flight of the target (compare Figs. 1(b) and 5). To reduce noise in the data due to GNSS drift of the UAV position, only the last $n_{\text{fit}[k]} \leq N_{\text{fit,max}}$ detections are considered ($N_{\text{fit,max}}$ is an empirically selected parameter). Additionally, the assumption that the trajectory is planar is leveraged to simplify the problem by projecting all detected positions to a plane, which is fitted to the data and constraining the line segments to this plane. The output of the algorithm is the lurking pose for the catcher UAV $W_{[k]} \equiv \{\mathbf{u}_{\text{lurk}[k]}, \varphi_{\text{lurk}[k]}\}$, where $\mathbf{u}_{\text{lurk}[k]}$ is the selected lurking position at time-step k and $\varphi_{\text{lurk}[k]}$ is the corresponding desired heading of the catcher UAV.

Definition. For the purposes of this section, a general line L is expressed as

$$L \equiv \{\mathbf{o}, \tilde{\mathbf{d}}\}, \quad \mathbf{u} = \mathbf{o} + s\tilde{\mathbf{d}}, \quad (1)$$

where $\mathbf{o} \in \mathbb{R}^3$ is the origin of the line, $\tilde{\mathbf{d}} \in \mathbb{R}^3$ is its direction, $\mathbf{u} \in \mathbb{R}^3$ is a general point on the line and $s \in \mathbb{R}$ is a line parameter.

Such a line may be converted to a line segment $\hat{L} \equiv \{\hat{\mathbf{o}}, \hat{\mathbf{d}}, \hat{l}\}$, $\hat{\mathbf{u}} = \hat{\mathbf{o}} + \hat{s}\hat{\mathbf{d}}$, based on a set of its inlier points $\mathcal{I} = \{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_{|\mathcal{I}|}\}$, using the following equations:

$$\hat{s}_{\min} = \min_{\mathbf{p}_i \in \mathcal{I}} \tilde{\mathbf{d}}^\top \cdot (\mathbf{p}_i - \mathbf{o}), \quad \hat{s}_{\max} = \max_{\mathbf{p}_i \in \mathcal{I}} \tilde{\mathbf{d}}^\top \cdot (\mathbf{p}_i - \mathbf{o}), \quad (2)$$

$$\hat{\mathbf{d}} = \tilde{\mathbf{d}} / \|\tilde{\mathbf{d}}\|, \quad \hat{l} = \hat{s}_{\max} - \hat{s}_{\min}, \quad (3)$$

$$\hat{\mathbf{o}} = \mathbf{o} + \hat{s}_{\min} \hat{\mathbf{d}}, \quad \hat{s} \in [0, \hat{l}]. \quad (4)$$

Here, $\tilde{\mathbf{d}}^\top \cdot (\mathbf{p}_i - \mathbf{o})$ corresponds to the distance between the projection of $\mathbf{p}_i$ to the line L and the line's origin $\mathbf{o}$, and $\hat{l}$ equals

to the length of the line segment $\hat{L}$. The line to line segment conversion is illustrated in Fig. 7.

The lurking pose $W_{[k]}$ is selected using a set of the last $n_{[k]}$ detected positions $S_{[k]} = \{b_{[k-n_{[k]}+1]}, b_{[k-n_{[k]}+2]}, \dots, b_{[k]}\}$. Steps of the selection algorithm (illustrated in Fig. 8) are:

1. The plane $P_{[k]}$ is fitted to the detected positions $S_{[k]}$ using the Reduced Heteroscedastic Errors-In-Variables (RHEIV) surface fitting algorithm [44].⁷
2. The positions $S_{[k]}$ are projected to the plane $P_{[k]}$ to obtain a set of projected positions $S'_{[k]}$.
3. A line $L_{1[k]}$ is fitted to $S'_{[k]}$ using RANSAC. Orientation of the vector $\vec{d}_{1[k]}$ describing the line is flipped if necessary to match the median orientation calculated from its inlier points $\mathcal{I}_{1[k]}$ (see Algorithm 3). A second line $L_{2[k]}$ is fitted to $S'_{[k]} \setminus \mathcal{I}_{1[k]}$ and oriented in the same manner as $L_{1[k]}$.
4. Lines $L_{1[k]}$ and $L_{2[k]}$ are converted to line segments $\hat{L}_{1[k]}$ and $\hat{L}_{2[k]}$ using their corresponding inlier point sets $\mathcal{I}_{1[k]}$ and $\mathcal{I}_{2[k]}$ and the previously described method.
5. A single line segment $\hat{L}_{\text{sel}[k]}$ is selected, according to the rule

$$\begin{aligned} \hat{L}_{\text{sel}[k]} &\equiv \{\hat{o}_{\text{sel}[k]}, \vec{d}_{\text{sel}[k]}, \hat{l}_{\text{sel}[k]}\}, \\ \hat{L}_{\text{sel}[k]} &= \underset{\hat{L}_{i[k]} \in \hat{\mathcal{L}}_{[k]}}{\operatorname{argmax}} \vec{d}_{i[k]} \cdot \vec{d}_{\text{sel}[k-1]}, \end{aligned} \quad (5)$$

where $\hat{\mathcal{L}}_{[k]} = \{\hat{L}_{1[k]}, \hat{L}_{2[k]}\}$. The initial direction vector is

defined as $\vec{d}_{\text{sel}[0]} = [\cos \varphi, \sin \varphi, 0]^\top$, where φ is a preferred heading of the lurking pose which is specified by the user. This rule selects the line segment with the smaller angle between its direction vector and the previously selected line segment, assuring consistency of the chosen lurking pose between consecutive time-steps.

6. The line segment $\hat{L}_{\text{sel}[k]}$ may stretch outside the target's linear segment of the trajectory if some points of the subsequent circular segment of the trajectory are selected as its inliers. To ensure that the resulting interception spot $\mathbf{u}_{\text{int}[k]}$ does not lie inside this incorrect region, it is selected in distance $s_{\text{step}[k]}$ from the end of oriented line segment $\hat{L}_{\text{sel}[k]}$. Specifically, it is selected as

$$\hat{s}_{\text{step}[k]} = \hat{l}_{\text{sel}[k]} - \min \left\{ \hat{s}_{\text{back}}, \frac{\hat{l}_{\text{sel}[k]}}{2} \right\}, \quad (6)$$

$$\mathbf{u}_{\text{lurk}[k]} = \hat{o}_{\text{sel}[k]} + \hat{s}_{\text{step}[k]} \vec{d}_{\text{sel}[k]}, \quad (7)$$

where $\hat{s}_{\text{back}}$ is the desired distance (determined empirically) of the point $\mathbf{u}_{\text{lurk}[k]}$ from the end of the line segment $\hat{L}_{\text{sel}[k]}$, and $\hat{s}_{\text{step}[k]}$ is the resulting distance clamped so that $\mathbf{u}_{\text{lurk}[k]}$ lies on the second half of $\hat{L}_{\text{sel}[k]}$.

7. The desired lurking heading $\varphi_{\text{lurk}[k]}$ is selected as orientation of the projection of $-\vec{d}_{\text{sel}[k]}$ to the XY plane of the static global coordinate frame. In other words, the heading is selected so that the catcher UAV is oriented in the direction opposite the estimated velocity of the target at

⁷ RHEIV is an iterative optimization algorithm for model-fitting using a maximum likelihood-based cost function, assuming a Gaussian model of errors of the input data. Unlike other fitting algorithms, such as mean squared error minimization or the previously mentioned RANSAC, RHEIV considers the uncertainty of the data (represented by a covariance matrix for each data point). This allows it to utilize additional information from the detection algorithm (not only the position estimate of the target $\mathbf{b}_{[i]}$, but also the respective covariance matrix $\mathbf{Z}_{\mathbf{b}[i]}$), which is important in order to fuse uncertain data without biasing the result.

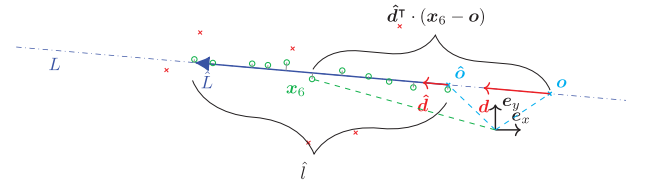


Fig. 7. Illustration of conversion of the line L to a line segment $\hat{L}$. Inlier points $\mathcal{I}$ of line L are marked by green circles, outliers $S \setminus \mathcal{I}$ by red crosses.

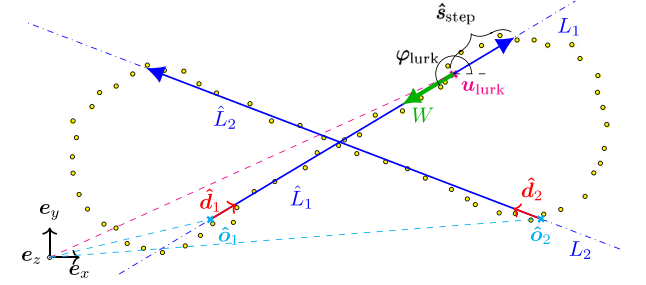


Fig. 8. Illustration of the lurking pose $W \equiv \{\mathbf{u}_{\text{lurk}}, \varphi_{\text{lurk}}\}$ selection method, described in Section 3.3.1 (top view of the same situation as in Fig. 5). Ball detections $\mathbf{b}_i$ from S are marked with yellow circles, line segments $\hat{L}_1$ and $\hat{L}_2$ with blue arrows, and the resulting lurking pose W with a green arrow. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

the point $\mathbf{u}_{\text{lurk}[k]}$, with the catching net pointing towards the approaching target in the next period of its trajectory (see Fig. 8).

To avoid producing an invalid result in case of an imprecisely fitted line $L_{1[k]}$ or $L_{2[k]}$, the result is discarded if any of the following conditions are true:

- the number of points for the plane fitting is too small (i.e. when $|S_{[k]}| < N_{\text{fit},\min}$),
- the angle between directional vectors of the lines is smaller than $\frac{\pi}{2}$ (i.e. $\vec{d}_{1[k]} \cdot \vec{d}_{2[k]} > 0$),
- the number of inlier points of any of the two lines is too small ($|\mathcal{I}_{1[k]}| < N_{\text{inl},\min} \vee |\mathcal{I}_{2[k]}| < N_{\text{inl},\min}$).

Algorithm 3 Algorithm for calculation of line orientation

```

1: Input:
2:  $\mathcal{I} = \{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_{|\mathcal{I}|}\}$   $\triangleright$  a set of inlier points of the line, sorted by their time-stamps
3:  $\vec{d} \in \mathbb{R}^3$   $\triangleright$  a direction vector of the line
4: Output:
5:  $\vec{d} \in \mathbb{R}^3$   $\triangleright$  an oriented direction vector of the line
6:  $\mathcal{O} := \emptyset$   $\triangleright$  initialize the set of relative orientations
7:  $\mathbf{p}_{\text{prev}} := \mathbf{p}_1$   $\triangleright$  initialize the previous position
8: for each  $i \in \{2, 3, \dots, |\mathcal{I}|\}$  do
9:  $\vec{d}' := \mathbf{p}_i - \mathbf{p}_{\text{prev}}$   $\triangleright$  estimate trajectory direction as difference of consecutive positions
10:  $o' := \text{sign}(\vec{d}^\top \cdot \vec{d}')$   $\triangleright$  calculate current relative orientation
11:  $\mathcal{O} := \mathcal{O} \cup o'$   $\triangleright$  add the current relative orientation to the set of orientations
12:  $\mathbf{p}_{\text{prev}} := \mathbf{p}_i$   $\triangleright$  update the previous position
13:  $o := \text{median}(\mathcal{O})$   $\triangleright$  estimate final relative orientation as median of  $\mathcal{O}$ 
14:  $\vec{d} := o \vec{d}$   $\triangleright$  flip the direction vector if necessary

```

3.3.2. Trajectory prediction

A linear Kalman Filter (KF) is used for predicting the future trajectory of the target. Dynamics of the target are modeled as a linear first order motion model of a point mass in 3D, although

the model could trivially be extended to higher-order dynamics, if the specific application requires. The state vector is defined as

$$\mathbf{x} = [\hat{\mathbf{b}}^\top \quad \vec{\hat{\mathbf{v}}}^\top]^\top, \quad (8)$$

where $\hat{\mathbf{b}}$ is an estimate of the target's position in the static global coordinate frame and $\hat{\mathbf{v}}$ is an estimate of its velocity. The transition and observation models are defined as a discrete stochastic linear system in the form

$$\mathbf{x}_{[k]} = \mathbf{A}_{[k]} \mathbf{x}_{[k-1]} + \boldsymbol{\xi}_{[k]}, \quad \boldsymbol{\xi}_{[k]} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Xi}_{[k]}), \quad (9)$$

$$\mathbf{z}_{[k]} = \mathbf{H}_{[k]}\mathbf{x}_{[k]} + \boldsymbol{\zeta}_{[k]}, \quad \boldsymbol{\zeta}_{[k]} \sim \mathcal{N}(\mathbf{0}, \mathbf{Z}_{[k]}). \quad (10)$$

The measurement vector $\mathbf{z}_{[k]}$ is defined as

$$\mathbf{z}_{[k]} = \begin{cases} [\mathbf{b}_{[k]}^\top \bar{\mathbf{v}}_{[k]}^\top]^\top, & \text{if } \bar{\mathbf{v}}_{[k]} \text{ is available,} \\ \mathbf{b}_{[k]}, & \text{otherwise,} \end{cases} \quad (11)$$

where $\mathbf{b}_{[k]}$ is the target ball's detected position and $\overline{\mathbf{v}}_{[k]}$ is the measured velocity at time-step k . Similarly, the measurement covariance is defined as

$$\mathbf{Z}_{[k]} = \begin{cases} \begin{bmatrix} \mathbf{Z}_{\mathbf{b}[k]} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{Z}_{\vec{\mathbf{v}}} \end{bmatrix}, & \text{if } \vec{\mathbf{v}}_{[k]} \text{ is available,} \\ \mathbf{Z}_{\mathbf{b}[k]}, & \text{otherwise,} \end{cases} \quad (12)$$

where $\mathbf{Z}_{b[k]}$ is the covariance matrix of the ball position at time-step k , $\mathbf{0}_{n \times n}$ is an $n \times n$ zero matrix, and $\mathbf{Z}_{\vec{v}}$ is a static diagonal covariance matrix of the target's velocity. $\mathbf{Z}_{b[k]}$, $\mathbf{b}_{[k]}$ and $\vec{v}_{[k]}$ are outputs of the detection and velocity estimation algorithm, described in Section 3.2. Covariance matrix $\Xi_{[k]}$ of the process noise $\xi_{[k]}$ is

$$\underline{\mathbf{E}}_{[k]} = \Delta t_{[k]} \hat{\underline{\mathbf{E}}}, \quad (13)$$

where $\Delta t_{[k]}$ is the duration since the last update of the KF ($\Delta t_{[k]} = t_{[k]} - t_{[k-1]}$) and $\hat{\Sigma}$ is a covariance matrix of the process noise normalized to a time-step of 1s. The covariance matrices $\mathbf{Z}_{\bar{v}}$ and Σ were tuned together with the parameters of $\mathbf{Z}_{\mathbf{b}}$ (see Section 3.2.3) with data from preliminary experiments and simulations to minimize the position and velocity estimation error and ensure stability of the estimate. The system matrices are

$$\mathbf{A}_{[k]} = \begin{bmatrix} \mathbf{I}_{3 \times 3} & \Delta^t[k] \mathbf{I}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{I}_{3 \times 3} \end{bmatrix}, \quad (14)$$

$$\mathbf{H}_{[k]} = \begin{cases} \mathbf{I}_{6 \times 6}, & \text{if } \vec{\mathbf{v}}_{[k]} \text{ is available,} \\ [\mathbf{I}_{3 \times 3} \ \mathbf{0}_{3 \times 3}], & \text{otherwise,} \end{cases} \quad (15)$$

where $\mathbf{I}_{n \times n}$ is an identity matrix with dimensions $n \times n$.

The KF state $\mathbf{x}$ and its covariance matrix $\mathbf{P}$ are updated each time a new detection is available. If no valid state estimate is available (either in the first iteration of the KF update step or because the previous one was invalid), the KF state is initialized using the last $n_{\text{init}[k]} \leq N_{\text{init}, \text{max}}$ measurements, which are not older than $\Delta t_{\text{init}, \text{max}}$, or left invalid if $k < n_{\text{init}[k]} + 1$ (not enough measurements are available yet).

To ensure consistency of the state estimate, the estimate is invalidated if any of the following conditions holds true:

- the number of initialization measurements is too small (i.e. $n_{\text{init}[k]} \leq n_{\text{init},\min}$),
- the velocity estimate is invalid ($\|\hat{\mathbf{v}}_{[k]}\| - s_{\text{gt}[k]} > \Delta s_{\text{max}}$),
- the last update was too long ago ($t_{[k]} - t_{[k-1]} > \Delta t_{\text{max}}$).

If a valid state estimate is available at time-step k , the future trajectory of the target is predicted with a time horizon t_{horizon} using Eq. (9) (assuming $\xi_{[t]} = \mathbf{0}$) and the state estimate $\mathbf{x}_{[k]}$. If a selected line segment $\hat{l}_{\text{sel}[k]}$ is available, and the current position estimate of the ball is closer to the line than d_{snap} , the velocity estimate $\hat{\mathbf{v}}_{[k]}$ is “snapped” to the line direction (i.e. $\hat{\mathbf{v}}_{[k]} := s_{\text{gt}[k]} \hat{\mathbf{d}}_{\text{sel}[k]}$). Similarly, if a valid plane fit $P_{[k]}$ is available, the

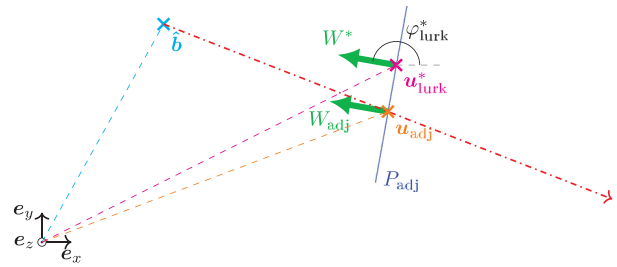


Fig. 9. Illustration of the lurking pose adjustment (top view). The predicted trajectory is marked with a red dash-dotted arrow.

predicted positions are “snapped” by projecting them to $P_{[k]}$. The snapping serves to make the predicted trajectory more robust by incorporating more of the available information into the prediction.

3.4. Online lurking pose adjustment

To account for lurking pose drift caused by GNSS inaccuracies as well as a potentially imprecise initial lurking pose selected using the algorithm from Section 3.3.1, the pose is adjusted online based on the latest prediction of the target's future trajectory. The target's trajectory is predicted by the algorithm described in Section 3.3.2. The pose is adjusted in the plane P_{adj} , defined by a normal vector $[\cos \varphi_{\text{lurk}}^*, \sin \varphi_{\text{lurk}}^*, 0]$ and the point $\mathbf{u}_{\text{lurk}}^*$. An intersection $\mathbf{u}_{\text{adj}[k]}$ of the predicted trajectory at time-step k and the plane P_{adj} is calculated. The new lurking pose is then $W_{\text{adj}[k]} \equiv \{\mathbf{u}_{\text{adj}[k]}, \varphi_{\text{lurk}}^*\}$. This process is illustrated in Fig. 9.

Note that the lurking orientation is not online-adjusted. This is done in order to avoid skewing the artifacts caused by a bad time-synchronization of points in the LiDAR scans when transforming the point-clouds to the static world-frame. The orientation error of the lurking position was negligible in practice (below 5 degrees) and was compensated for with a sufficiently wide catching net. The online pose adjustment algorithm can be modified trivially to account for an orientation adjustment, if required.

4. Experimental evaluation

4.1. Simulated experiments

Simulated experiments with a known ground-truth position of the target were executed to evaluate the precision and general performance of the detection algorithm. The setup of the experiments mimicked the competition scenario, while the realistic Gazebo simulator⁸ was used to ensure high fidelity of the simulation. A drone with the target ball suspended beneath it on a 1.45 m rod followed a randomly oriented and positioned eight-shaped trajectory with similar dimensions, position, and orientation as seen in the competition. A UAV equipped with a LiDAR sensor observed the target, and the detection and trajectory estimation methods described in Section 3 were used to localize it.

A total of 5500 measurements from the simulations were compared with the corresponding ground truth positions of the target. Fig. 10 shows the average recall and localization error over distance of the target from the sensor. The recall is calculated as

$$\text{recall}(d) = \frac{\text{TP}(d)}{\text{TP}(d) + \text{FN}(d)}, \quad (16)$$

8 gazeboosim.org.

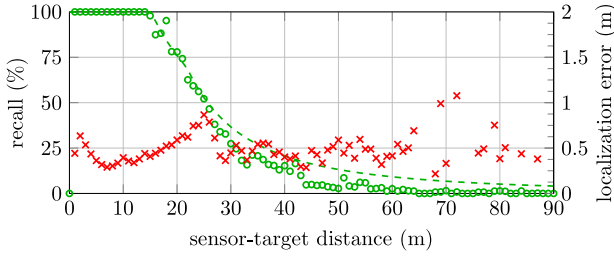


Fig. 10. Detection recall and localization error over distance of the target from the sensor (data from simulations). The recall is marked with green circles, and the localization error with red crosses. Theoretical probability of detection of the target is highlighted with a green dashed line.

Table 3

Comparison of the presented solution (bold) with our previously published methods for marker-less drone relative localization [1,2], and two other detection approaches from Challenge 1 of MBZIRC 2020 [15,17].

Detection method	avg. error	max. range
LiDAR-based (drone + ball)	0.44 m	88 m
depthmap-based (drone only) [1]	1.91 m	50 m
CNN-based (drone only) [2]	3.76 m	32 m
CNN-based (balloons) [15]	not reported	50 m
CNN-based (drone + ball) [17]	not reported	~50 m

where d is the distance of the target from the sensor, $TP(d)$ is the number of True Positive detections at distance d , and $FN(d)$ is the number of False Negatives at distance d . The distance d is discretized into bins with width of 1 m for the purposes of these calculations. The total average localization error was 0.44 m and the maximal detection distance was 87.79 m. A comparison with two different methods for marker-less drone relative localization (one utilizing a depth-based detection algorithm, and the other based on a Convolutional Neural Network (CNN) processing RGB images) is presented in Table 3. Additionally, a comparison with two detection methods used by other teams at the MBZIRC 2020 Competition is provided in the table. The values in Table 3 demonstrate that, when applied to a similar domain, our detector provided better range and precision than computer vision methods.

An estimate of the theoretical probability of detection over distance was calculated using known dimensions of the target and parameters of the LiDAR sensor, based on the assumption that the target's position is distributed uniformly in the FOV of the sensor. The probability was estimated as the combined probability of at least one vertical sample of the LiDAR sensor intersecting the target (p_{ver}), and at least three horizontal samples intersecting the target (p_{hor}):

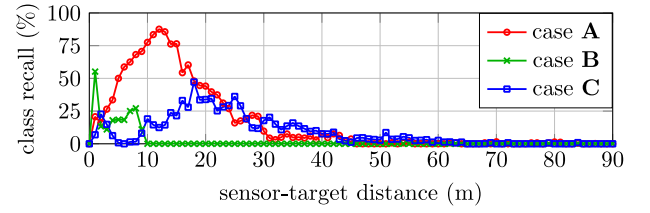
$$p_{ver}(d) = \min \left\{ \frac{\arctan(\frac{h_{tgt}}{2d})}{\alpha}, 1 \right\}, \quad (17)$$

$$p_{hor}(d) = \min \left\{ \frac{\arctan(\frac{w_{tgt}}{2d})}{3\beta}, 1 \right\}, \quad (18)$$

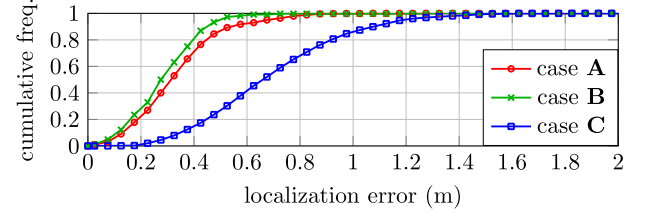
$$p(d) = p_{ver}(d) \cdot p_{hor}(d), \quad (19)$$

where d is the distance of the target, h_{tgt} and w_{tgt} are the target's dimensions, and α and β are the vertical and horizontal angular resolutions of the LiDAR. The estimated probability is shown in Fig. 10. It may be observed that the measured recall of the detection algorithm fits the theoretical detection probability well, suggesting that under the current assumptions, the recall cannot be significantly improved.

Class-specific recalls for the three valid cases of target classification (see Section 3.2.2) are in Fig. 11(a). The class-specific recall



(a) Class-specific recalls over distance of the target from the sensor.



(b) Cumulative frequency of localization error for the different classified cases.

Fig. 11. Evaluation of detection performance of the different classes based on the simulated experiments.

at distance d for the case **A** is calculated as

$$\text{recall}_A(d) = \frac{TP_A(d)}{TP(d) + FN(d)}, \quad (20)$$

where $TP_A(d)$ is the number of True Positives classified as case **A** at distance d . Class-specific recall for the other cases is calculated analogously. Fig. 11(b) shows class-specific localization errors as a cumulative frequency (estimate of the cumulative probability distribution). The recalls and localization errors for the different cases correspond well to the expected results, as discussed in Sections 3.2.2 and 3.2.3.

4.2. Competition results

Data from the competition was examined to evaluate the performance of the presented interception method. Photos from the preliminary experiments and from the competition are in Fig. 12. A video summary of one competition trial, as well as other videos related to this work, are available online.⁹

Our solution scored first place in the Grand Challenge and second place in the separate Challenge 1. The target ball was successfully knocked off the carrier drone a total of three times during the competition (twice in the first trial of Challenge 1 and once in the Grand Challenge). However, we did not manage to deliver the ball to the designated landing position, which is why our team did not score the full 100 points (see Table 4). This was due to a design flaw with the catching net, which was unable to catch and hold the ball after it was detached. Instead, the ball was knocked away from the net and fell to the ground (this is illustrated in the video footage available online⁹). Because physical interception is a very risky manoeuvre and the hardware platforms and sensors involved are expensive, we did not test the final manoeuvre before the competition, so as to avoid damaging the UAV and LiDAR. Therefore, we did not have the opportunity to iterate and correct the net design.

To evaluate the precision of the interception, a ground-truth interception position was obtained using linear interpolation between the closest ball positions before and after the ball passed around the UAV (see Fig. 13). In an ideal case, this ground-truth

⁹ mrs.felk.cvut.cz/mbzirc-2020-ball-catching.

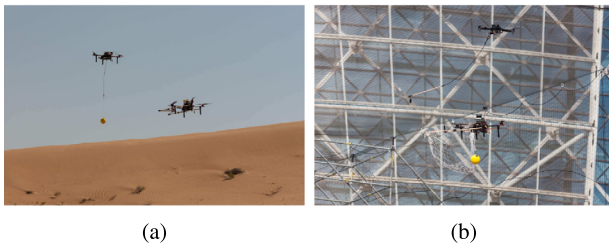


Fig. 12. Photos from the experimental evaluation of the presented method in preliminary experiments in a desert near Abu Dhabi (12(a)) and in one of the competition trials in Abu Dhabi (12(b)).

Table 4

Score of the top 3 teams in the Challenge 1 part of the Grand Challenge and in the separate Challenge 1. The better result of the two trials in the Challenge 1 was counted according to the competition rules. The maximum score was 100. The row corresponding to our team is highlighted in bold. DNQ stands for Did Not Qualify.

Team	G. Ch.	Ch. 1
CTU in Prague, UPENN and NYU	72	72
UPM, UPO, PUT and CNRS	40.5	18
University of Bonn	30	30
University of Tokyo	0	42
Beijing Institute of Technology	DNQ	100

position would be equal to the prediction of the interception point $\mathbf{u}_{adj}$ (see Section 3.4) and the physical UAV position during the manoeuvre. Euclidean distance of the predicted interception point $\mathbf{u}_{adj}$ from the ground-truth interception position was calculated for each trajectory period (pass) of the carrier drone. This distance indicates the error of the detection and prediction algorithms. Similarly, the distance of the interceptor's physical position from the ground-truth was calculated. When compared to the prediction error, the error caused by the UAV's motion dynamics and control algorithms is visible. The UAV's closest physical distance from the actual position of the ball was the determining factor for a successful or failed interception. Only the ball trajectory periods when the **Lurk** state of the mission control state machine (when the UAV was adapting its position to catch the target, see Section 3.1) was active were considered for the evaluation.

The resulting values are presented in Table 5. Graphs of the errors over time during several passes of the ball are seen in Fig. 14. From the graphs, it is evident that there is consistently a spike in the prediction error a few seconds before the interception, which causes the catcher UAV to temporarily diverge from the desired position. After examination of the collected data, we have concluded that this was caused by the ball swinging due to a change in the curvature of the ball's trajectory when it transitions from the circular segment of the trajectory to the linear segment. Even though the ball swing is neglected in our model of the ball's trajectory, our system still managed to successfully intercept the ball multiple times.

The average error of the catcher UAV position from the ball pass-through during the competition was 1.21 m. The average error of the predicted interception point was 0.86 m. The error of the predicted interception position is smaller due to the UAV position lagging behind the position setpoint, which sometimes caused the interception to fail even when the prediction was precise (see Table 5). The data suggests that the ball was intercepted successfully when the distance between the center of the catcher UAV and the ball position was smaller than approx. 0.7 m. This roughly corresponds to the size of the catcher UAV, which was 1.2 m × 1.2 m × 0.5 m.

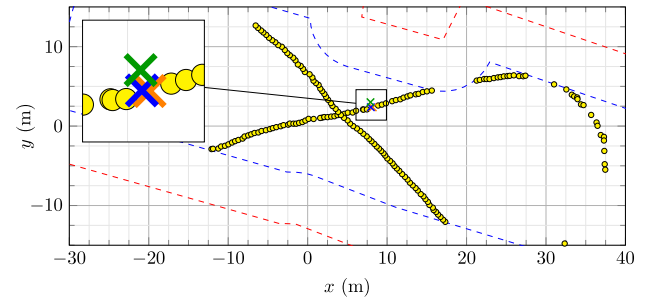


Fig. 13. Top view of the interception situation during the fourth pass of the target ball (corresponding to the sixth period of the ball trajectory) in the first trial. Detections are marked with yellow dots. Position of the UAV when the ball was passing through the interception position is marked with a green cross, position of the ball with an orange cross, and its predicted position with a blue cross. Position error of the UAV during the interception was 0.75 m and error of the ball's predicted position was 0.25 m in this pass (see Table 5). The operational area is marked with a blue dashed line and the safety area with a red dashed line. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 5

Error of the UAV position in the different passes (in meters). Values in parentheses are errors of the corresponding predicted intersection points. Last column contains times of interception since takeoff of the catcher UAV. Values corresponding to the successful catch are in bold.

Pass	1	2	3	4	5	6	tot. time
Ch. 1	1.12 (1.55)	1.99 (1.32)	0.71 (0.56)	0.75 (0.25)	1.35 (0.97)	0.62 (0.23)	123 s
G. Ch.	1.82 (0.77)	1.40 (1.40)	1.36 (1.31)	1.44 (1.12)	0.69 (0.69)	N/A	229 s

5. Going beyond MBZIRC

The main motivation of the MBZIRC 2020 Challenge 1 was to further the research in drone intrusion prevention and autonomous intruder interception. Our team is designing a prototype autonomous aerial interception system (AAIS) *Eagle.One*,¹⁰ which is the result of several years of our group's research in these areas. The experience and inspiration from the MBZIRC competition were integrated into the *Eagle.One* platform, leading to changes and enhancements of the target detection and tracking algorithms, as well as hardware changes of the platform (see Fig. 15). Namely, a novel method for detecting flying objects using a LiDAR based on the algorithms presented in this paper is being developed. Comparison with other related drone detection methods suggests that a LiDAR-based detector can provide better precision and range when applied to a similar domain, as is discussed in Section 4. The results of the new detector show improvements over the previously developed methods [1,2], in both precision and a decrease of false negatives.

6. Conclusion

We have provided a detailed description and evaluation of our solution to the MBZIRC 2020 Challenge 1 ball catching task. The tasks in the MBZIRC competition are designed to be difficult and require development of novel perception, control, and planning methods with the aim of expanding research in related robotic fields. After examination of the ball catching problem in simulations and preliminary experiments, we have developed a LiDAR-based algorithm for the detection of a flying target in a designated

¹⁰ eagle.one.

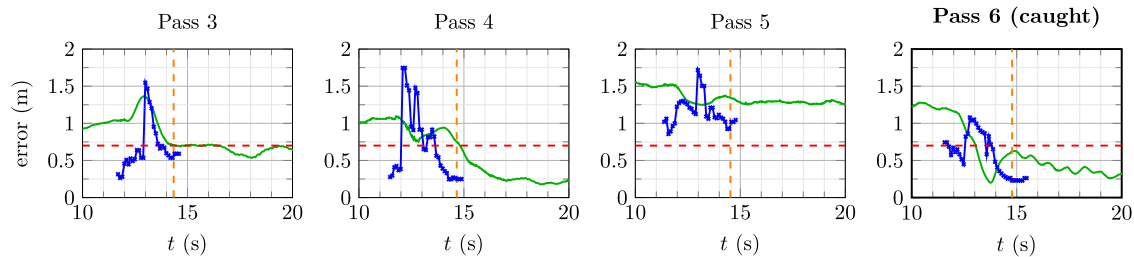


Fig. 14. Graphs of the UAV position error (green) and the predicted interception position error (blue) over time in the last four passes of the target ball in the first trial of Challenge 1. The interception time (when the ball was closest to the UAV) is marked with a vertical orange dashed line. The empirically determined position error threshold for a successful interception (0.7 m) is marked with a horizontal red dashed line. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)



Fig. 15. A photo of the updated *Eagle.One* Mk 2 autonomous aerial interception system in-flight, while tracking a flying DJI Mavic Air drone. The *Eagle.One* Mk 2 prototype now utilizes the same LiDAR sensor as was used in the MBZIRC competition. The detection and tracking method is an evolution of the algorithms presented in this paper.

area and complementary trajectory prediction and interception planning algorithms. The presented methods proved to be robust and precise, as is shown in the experimental evaluation. Our system placed first in the Grand Challenge and second in Challenge 1 of the competition among 22 competing expert teams, demonstrating a beyond-state-of-the-art performance. Moreover, the newly developed algorithms and experience gained have inspired further evolution and improvement of an autonomous aerial interception system to whose research and development we are contributing, thus fulfilling the main purpose of this competition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We thank the University of Pennsylvania and the New York University for the collaboration as the members of the CTU-UPenn-NYU team. The presented work was supported mainly by the Czech Science Foundation (GAČR) under research project No. 20-10280S and by the Khalifa University grant MBZIRC 2020. Some parts were co-funded by CTU grant no SGS20/174/OHK3/3T/13, OP VVV funded project CZ.02.1.01/0.0/0.0/16 019/0000765 “Research Center for Informatics” and EU H2020 project AERIAL-CORE, No. 871479.

References

- [1] M. Vrba, D. Heřt, M. Saska, Onboard marker-less detection and localization of non-cooperating drones for their safe interception by an autonomous aerial system, *IEEE Robot. Autom. Lett.* 4 (4) (2019) 3402–3409, <http://dx.doi.org/10.1109/LRA.2019.2927130>.

- [2] M. Vrba, M. Saska, Marker-less micro aerial vehicle detection and localization using convolutional neural networks, *IEEE Robot. Autom. Lett.* 5 (2) (2020) 2459–2466, <http://dx.doi.org/10.1109/LRA.2020.2972819>.
- [3] M. García, A. Viguria, G. Heredia, A. Ollero, Minimal-time trajectories for interception of malicious drones in constrained environments, in: *ICVS*, 2019, pp. 734–743.
- [4] N. Souli, R. Makrigiorgis, A. Anastasiou, A. Zacharia, P. Petrides, A. Lazanas, P. Valianti, P. Kolios, G. Ellinas, HorizonBlock: Implementation of an autonomous counter-drone system, in: *ICUAS*, 2020, pp. 398–404, <http://dx.doi.org/10.1109/ICUAS48674.2020.9213871>.
- [5] M. García, R. Caballero, F. González, A. Viguria, A. Ollero, Autonomous drone with ability to track and capture an aerial target, in: *ICUAS*, 2020, pp. 32–40, <http://dx.doi.org/10.1109/ICUAS48674.2020.9213883>.
- [6] K.-H. Zeng, R. Mottaghi, L. Weihs, A. Farhadi, Visual reaction: Learning to play catch with your drone, 2019, arXiv e-prints [arXiv:1912.02155](https://arxiv.org/abs/1912.02155).
- [7] K. Su, S. Shen, Catching a flying ball with a vision-based quadrotor, in: *ISER*, 2017, pp. 550–562.
- [8] R. Ritz, M.W. Müller, M. Hehn, R. D’Andrea, Cooperative quadcopter ball throwing and catching, in: *IROS*, 2012, pp. 4972–4978.
- [9] E. Mueggler, N. Baumli, F. Fontana, D. Scaramuzza, Towards evasive maneuvers with quadrotors using dynamic vision sensors, in: *ECMR*, 2015, pp. 1–8.
- [10] A.D. de Quevedo, F.I. Urzaiz, J.G. Menoyo, A.A. Lopez, Drone Detection With X-Band Ubiquitous Radar, in: *IRS*, 2018, pp. 1–10.
- [11] J. Park, S. Park, D. Kim, S. Park, Leakage mitigation in heterodyne FMCW radar for small drone detection with stationary point concentration technique, *IEEE Trans. Microw. Theory Tech.* 67 (3) (2019) 1221–1232.
- [12] J. Busset, F. Perrodin, P. Wellig, B. Ott, K. Heutschi, T. Rühl, T. Nussbaumer, Detection and tracking of drones using advanced acoustic cameras, in: *Unmanned/Unattended Sensors and Sensor Networks XI*; and *Advanced Free-Space Optical Communication Techniques and Applications*, Vol. 9647, 2015, pp. 53–60.
- [13] Z. Shi, X. Chang, C. Yang, Z. Wu, J. Wu, An acoustic-based surveillance system for amateur drones detection and localization, *IEEE Trans. Veh. Technol.* 69 (3) (2020) 2731–2739.
- [14] F. Svanstrom, C. Englund, F. Alonso-Fernandez, Real-time drone detection and tracking with visible, thermal and acoustic sensors, 2020, arXiv e-prints [arXiv:2007.07396](https://arxiv.org/abs/2007.07396).
- [15] M. Beul, S. Bultmann, A. Rochow, R.A. Rosu, D. Schleich, M. Splietker, S. Behnke, Visually guided balloon popping with an autonomous MAV at MBZIRC 2020, in: *SSRR*, 2020, pp. 34–41, <http://dx.doi.org/10.1109/SSRR50563.2020.9292612>.
- [16] L.A. Tony, S. Jana, V. BV, M.S. Gadde, A. Kashyap, R. Ravichandran, D. Ghose, et al., Collaborative tracking and capture of aerial object using UAVs, 2020, arXiv e-prints [arXiv:2010.01588](https://arxiv.org/abs/2010.01588).
- [17] M. Zhao, T. Anzai, F. Shi, T. Maki, T. Nishio, K. Ito, N. Kuromiya, K. Okada, M. Inaba, Versatile multilinked aerial robot with tilting propellers: Design, modeling, control and state estimation for autonomous flight and manipulation, 2020, arXiv e-prints [arXiv:2008.05613](https://arxiv.org/abs/2008.05613).
- [18] M. Saqib, S.D. Khan, N. Sharma, M. Blumenstein, A study on detecting drones using deep convolutional neural networks, in: *AVSS*, 2017, pp. 1–5, <http://dx.doi.org/10.1109/AVSS.2017.8078541>.
- [19] A. Schumann, L. Sommer, J. Klatte, T. Schuchert, J. Beyerer, Deep cross-domain flying object classification for robust UAV detection, in: *AVSS*, 2017, pp. 1–6, <http://dx.doi.org/10.1109/AVSS.2017.8078558>.
- [20] C. Aker, S. Kalkan, Using deep networks for drone detection, in: *AVSS*, 2017, pp. 1–6, <http://dx.doi.org/10.1109/AVSS.2017.8078539>.
- [21] K.R. Sapkota, S. Roelofsen, A. Rozantsev, V. Lepetit, D. Gillet, P. Fua, A. Martinoli, Vision-based unmanned aerial vehicle detection and tracking for sense and avoid systems, in: *IROS*, 2016, pp. 1556–1561.
- [22] J. Li, D.H. Ye, T. Chung, M. Kolsch, J. Wachs, C. Bouman, Multi-target detection and tracking from a single camera in unmanned aerial vehicles (UAVs), in: *IROS*, 2016, pp. 4992–4997, <http://dx.doi.org/10.1109/IROS.2016.7759733>.

- [23] T. Jiang, D. Lin, T. Song, Vision-based autonomous landing of a quadrotor using a gimbaled camera, *Proc. Inst. Mech. Eng. G* 233 (14) (2019) 5093–5106, <http://dx.doi.org/10.1177/0954410019837777>.
- [24] A. Carrio, J. Tordesillas, S. Vemprala, S. Saripalli, P. Campoy, J.P. How, Onboard detection and localization of drones using depth maps, *Access* 8 (2020) 30480–30490.
- [25] M. Hammer, M. Hebel, B. Borgmann, M. Laurenzis, M. Arens, Potential of lidar sensors for the detection of UAVs, in: *Laser Radar Technology and Applications XXIII*, Vol. 10636, 2018, pp. 39–45, <http://dx.doi.org/10.1117/12.2303949>.
- [26] B. Kim, D. Khan, C. Bohak, W. Choi, H. Lee, M. Kim, V-RBNN based small drone detection in augmented datasets for 3D LADAR system, *Sensors* 18 (11) (2018) 3825, <http://dx.doi.org/10.3390/s18113825>.
- [27] M. Hammer, B. Borgmann, M. Hebel, M. Arens, UAV Detection, tracking, and classification by sensor fusion of a 360° lidar system and an alignable classification sensor, in: *Laser Radar Technology and Applications XXIV*, Vol. 11005, 2019, pp. 99–108.
- [28] T. Sankey, J. Donager, J. McVay, J.B. Sankey, UAV lidar and hyperspectral fusion for forest monitoring in the southwestern USA, *Remote Sens. Environ.* 195 (2017) 30–43, <http://dx.doi.org/10.1016/j.rse.2017.04.007>.
- [29] N. Bolourian, M.M. Soltani, A. Albahri, A. Hammad, High level framework for bridge inspection using LiDAR-equipped UAV, in: *ISARC*, 2017, pp. 683–688, <http://dx.doi.org/10.22260/ISARC2017/0095>.
- [30] J. Zhang, S. Singh, Low-drift and real-time lidar odometry and mapping, *Auton. Robots* 41 (2) (2017) 401–416.
- [31] M.U. de Haag, C.G. Bartone, M.S. Braasch, Flight-test evaluation of small form-factor LiDAR and radar sensors for sUAS detect-and-avoid applications, in: *DASC*, 2016, pp. 1–11.
- [32] Z. Li, C. Meng, F. Zhou, X. Ding, X. Wang, H. Zhang, P. Guo, X. Meng, Fast vision-based autonomous detection of moving cooperative target for unmanned aerial vehicle landing, *J. Field Robot.* 36 (1) (2019) 34–48.
- [33] T. Báča, P. Štěpán, V. Spurný, D. Heřt, R. Pěnička, M. Saska, J. Thomas, G. Loianno, V. Kumar, Autonomous landing on a moving vehicle with an unmanned aerial vehicle, *J. Field Robot.* 36 (2019) 874–891.
- [34] M. Beul, M. Nieuwenhuisen, J. Quenzel, R.A. Rosu, J. Horn, D. Pavlichenko, S. Houben, S. Behnke, Team NimbRo at MBZIRC 2017: Fast landing on a moving target and treasure hunting with a team of micro aerial vehicles, *J. Field Robot.* 36 (1) (2019) 204–229.
- [35] T. Báča, D. Heřt, G. Loianno, M. Saska, V. Kumar, Model predictive trajectory tracking and collision avoidance for reliable outdoor deployment of unmanned aerial vehicles, in: *IROS*, 2018, pp. 6753–6760.
- [36] M. Petrlik, T. Báča, D. Heřt, M. Vrba, T. Krajník, M. Saska, A robust UAV system for operations in a constrained environment, *IEEE Robot. Autom. Lett.* 5 (2) (2020) 2169–2176.
- [37] T. Báča, M. Petrlik, M. Vrba, V. Spurný, R. Penicka, D. Hert, M. Saska, The MRS UAV system: Pushing the frontiers of reproducible research, real-world deployment, and education with autonomous unmanned aerial vehicles, *J. Intell. Robot. Syst.* 102 (26) (2021) 1–28.
- [38] T. Lee, M. Leok, N.H. McClamroch, Geometric tracking control of a quadrotor UAV on SE(3), in: *CDC*, 2010, pp. 5420–5425.
- [39] M. Quigley, K. Conley, B.P. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, A.Y. Ng, ROS: an open-source Robot Operating System, in: *ICRA Workshop on Open Source Software*, 2009.
- [40] J. Zhang, S. Singh, LOAM: Lidar odometry and mapping in real-time, in: *Robotics: Science and Systems*, Vol. 2, 2014, pp. 1–9.
- [41] Y.-C. Lin, Y.-T. Cheng, T. Zhou, R. Ravi, S.M. Hasheminasab, J.E. Flatt, C. Troy, A. Habib, Evaluation of UAV LiDAR for mapping coastal environments, *Remote Sens.* 11 (24) (2019) 2893.
- [42] H. Qin, Z. Meng, W. Meng, X. Chen, H. Sun, F. Lin, M.H. Ang, Autonomous exploration and mapping system using heterogeneous UAVs and UGVs in GPS-denied environments, *IEEE Trans. Veh. Technol.* 68 (2) (2019) 1339–1350.
- [43] S. Ramasamy, R. Sabatini, A. Gardi, J. Liu, LIDAR obstacle warning and avoidance system for unmanned aerial vehicle sense-and-avoid, *Aerosp. Sci. Technol.* 55 (2016) 344–358.
- [44] W. Chojnacki, M.J. Brooks, A. van den Hengel, D. Gawley, FNS and HEIV: relating two vision parameter estimation frameworks, in: *ICIA*, 2003, pp. 152–157.



Matouš Vrba received his M.Sc. degree at the Czech Technical University in Prague, Czech Republic, and currently is pursuing a Ph.D. on computer vision methods for mutual localization of Unmanned Aerial Vehicles as a member of the Multi-robot Systems lab in CTU Prague since 2018. He was also a member of CTU-UPENN-NYU team in the MBZIRC 2020, and the CTU-CRAS-NORLAB team in the DARPA SubT competition.



Yurii Stasinchuk has received his bachelor degree at Ukrainian catholic university in Lviv, Ukraine and currently is pursuing master's degree at Czech technical university in Prague, Czech Republic. Since 2019, he is a part of the Multi-robot Systems lab in CTU Prague. He was also a member of CTU-UPENN-NYU team in the MBZIRC 2020, and the CTU-CRAS-NORLAB team in the DARPA SubT competition.



Tomáš Báča received his M.Sc. degree at the Czech Technical University in Prague, Czech Republic, and currently is pursuing a Ph.D. on distributed remote sensing. Since 2014, he is a part of the Multi-robot Systems lab in CTU Prague. Tomas focuses on distributed radiation sensing with unmanned aerial vehicles and planning and control for unmanned aerial vehicles. He was a key member of CTU-UPenn-UoL and CTU-UPENN-NYU teams in the MBZIRC 2017 and 2020 robotic competitions in Abu Dhabi.



Vojtěch Spurný received his M.Sc. degree at the Czech Technical University in Prague, Czech Republic, and currently is pursuing a Ph.D. on methods of planning and coordination for Unmanned Aerial Vehicles. Since 2014, he is a part of the Multi-robot Systems lab in CTU Prague. He was also a CTU-UPenn-UoL and CTU-UPENN-NYU team member in the MBZIRC 2017 and 2020 robotic competitions in Abu Dhabi, United Arab Emirates.



Matěj Petrlik received his M.Sc. degree at the Czech Technical University in Prague, Czech Republic, and currently is pursuing a Ph.D. on sensor fusion, co-operative mapping, and dynamic state estimation of Unmanned Aerial Vehicles. He was also a member of CTU-UPenn-UoL and CTU-UPENN-NYU teams in the MBZIRC 2017 and 2020, and the CTU-CRAS-NORLAB team in the DARPA SubT competition.



Daniel Heřt is currently working at the Multi-robot Systems lab in CTU Prague as a research assistant and a hardware specialist. He received his M.Sc. degree at the Czech Technical University in Prague, Czech Republic. Dan is focusing on control, planning, estimation, and hardware implementations for Unmanned Aerial Vehicles. He is a vital member of the MRS team by working on the maintenance and development of the diverse fleet of Unmanned Aerial Vehicles.



David Žaitlík received his M.Sc. degree at the Czech Technical University in Prague, Czech Republic, and currently is a research fellow focused on embedded systems. Since 2019, he is a part of the Multi-robot Systems lab in CTU Prague. He was a member of the CTU-UPENN-NYU team in the MBZIRC 2020 competition.



Martin Saska received his M.Sc. degree at Czech Technical University in Prague, 2005, and his Ph.D. degree at the University of Wuerzburg, Germany. Since 2009, he is a research fellow at Czech Technical University in Prague, where he has founded and since leads the Multi-robot Systems lab and co-founded Center for Robotics and Autonomous Systems. He was a visiting scholar at the University of Illinois at Urbana-Champaign (USA) in 2008, and at the University of Pennsylvania (USA) in 2012, 2014, and 2016. He is an author or co-author of more than 70 publications in peer-reviewed conferences and more than 20 publications in impacted journals.